NyayaSim (AI Court Sim): Tech Stack and Free Resources

The tools NyayaSim uses, how each one is used, and where to get every one of them for free. It is written for a team of first-year students building the MVP in one to two months. The companion documents cover the product requirements, technical requirements, database schema and workflow.

1. The stack at a glance

Every tool below is free. Where your Tech Stack Options document gave four choices, this list takes Option 1 and removes anything the MVP does not need.

Layer	Tool we use	Cost
Language	Python 3.11 or newer	Free
Coding assistant (replaces VS Code)	OpenCode, an open-source AI coding agent	Free
Models inside OpenCode	DeepSeek and GLM (free routes listed in Section 7)	Free routes, with limits
Teamwork and backup	Git and GitHub (private repository)	Free
Model for the running NyayaSim app (main)	Google Gemini API, free tier, key from Google AI Studio	Free, with limits
Model for the running app (backup)	Groq API, free tier. DeepSeek or GLM can also be used through the same gateway	Free, with limits
Checking model output	Pydantic	Free
Agent coordination	Plain Python functions (no agent framework)	Free
Database and logs	SQLite (built into Python)	Free
Legal corpus	A JSON file of numbered sections	Free
Interface	Streamlit	Free
Testing	pytest and the team's own scoring	Free
Secrets	A .env file kept out of Git	Free

2. How each tool is used

Tool	What it does in NyayaSim	Who uses it most
Python	All agents, rules, lookup, logging and scoring are written in it. One language for the whole team.	Everyone
OpenCode	Where the team writes, explains and fixes code. You describe a task in plain English, and it reads the project folder, edits files and runs commands, asking permission first. Section 3 explains how we use it.	Everyone
DeepSeek and GLM models	The AI brains behind OpenCode. DeepSeek V4 Flash for everyday coding; DeepSeek V4 Pro or GLM models for harder tasks.	Everyone
Git and GitHub	Stores the project, keeps history, and lets four people work without overwriting each other. Changes go in through reviewed pull requests.	Everyone
Gemini API	The AI that NyayaSim itself calls when it runs: it reads case text and writes judgments for the intake agent, procedural checker, citation support check and reviewer.	Agent engineer
Groq API	Second provider. Used if Gemini limits block you, and to compare model quality on your own test cases.	Data and verification engineer
Pydantic	Defines the exact shape of every agent's input and output. A badly shaped AI answer is rejected and retried. This is the typed boundary in your TRD.	Data and verification engineer
Plain Python orchestrator	Runs the agents in order, applies the early-stop rule and the escalation rule. Easy to read, debug and explain to examiners.	Agent engineer
SQLite	Saves every run and every agent step with its input and output, so any statement in a report can be traced.	Data and verification engineer
JSON legal corpus	Holds the chosen Consumer Protection Act, 2019 sections with unique IDs. The verifier checks every citation against it by exact ID.	Legal and product lead
Streamlit	The simple web page where a user pastes a case, runs the review and reads the report. Shows the fictional-data notice.	Evaluation and documentation lead
pytest	Runs quick checks on the code-only parts (rules, lookup, scoring) and the evaluation on test cases.	Evaluation and documentation lead
Environment file	Holds the API keys and the model name. It is never uploaded to GitHub.	Everyone

3. How we use OpenCode with GLM and DeepSeek

OpenCode is an open-source AI coding assistant that runs in the terminal (it also has a desktop app). It has no model of its own: you connect it to one. We connect it to DeepSeek and GLM, which are free or low cost for coding. Note that there are **two separate model jobs** in this project, and they should not be confused.

Job	Tool	Model
Helping students write and fix the project code	OpenCode	DeepSeek and GLM
Running the NyayaSim agents (intake, checker, citation support check, reviewer)	The model gateway inside the project	Gemini free tier by default, Groq as backup. DeepSeek or GLM can be used here too, because the gateway can be switched, but test them on your own cases first

Which models, and what they cost

Route	Models	Cost	Use it for
1. OpenCode's own free models	DeepSeek V4 Flash (free version) and other free models in the OpenCode Zen list	Free. One source lists about 200 free requests a day; check the current limit	Daily coding. Start here
2. Z.ai free tier	GLM-4.7-Flash and GLM-4.5-Flash, free for registered accounts	Free, with rate limits. The newer GLM-5 family is paid	A second free option when route 1 hits its limit
3. Low-cost paid (optional)	DeepSeek V4 Flash or Pro by token; OpenCode Go (about 10 dollars a month) with DeepSeek and GLM-5 models; GLM Coding Plan from about 18 dollars a month	Not free	Only if the team agrees to spend, for example near the deadline

The free list changes often. Check opencode.ai and the Z.ai pricing page before you plan around any model name.

How the team sets it up

1. Install OpenCode from opencode.ai. If you use the npm method, install Node.js from nodejs.org first.
2. Open a terminal inside the NyayaSim project folder and start OpenCode there.
3. Use OpenCode's connect command to add a provider. For route 1 sign in with OpenCode; for route 2 create a free Z.ai account and paste its API key.
4. Use OpenCode's models command to pick a model, for example DeepSeek V4 Flash (free). Change it later without losing your work.
5. Ask a simple question such as 'Explain what this folder is for' to confirm it works.
6. Keep the API keys only in your own environment file or in OpenCode's own sign-in. Never type a key into the chat.

Working safely with an AI coding assistant

- **One small task at a time.** For example: 'Explain this error in simple words' or 'Create the checklist file for consumer cases using the format in our PRD'.
- **Read every change before you keep it.** Every team member must be able to explain what each file does. Your examiners may ask.
- **Commit to Git after each working step**, so any bad change can be undone.
- **Test everything.** Free models are less reliable than top paid models and can invent a library feature that does not exist.

- **Let the assistant explain first** when you do not understand something, and ask for changes second.
- **Never give it real case data or keys.** Hosted models may store prompts or use them to improve the model. Fictional data only.
- **Keep the plan in your own hands.** The legal lead decides the checklist and ground truth. The assistant helps with code, not with legal judgments.

4. How the running app connects to its model

This section is about the model NyayaSim calls while it runs, not the model inside OpenCode. Connecting a model means your program sends it text and receives text back. NyayaSim keeps this simple and safe with one rule: **every AI call goes through a single gateway module**. Agents never call the model themselves.

1. Create a free API key in Google AI Studio and store it in the environment file.
2. Copy the name of a current Flash model from AI Studio into the environment file. Never write the model name inside agent code, because names change.
3. Each agent has its own short instruction (prompt) with one job, a list of rules, and a line saying the case text is data and not instructions.
4. The gateway sends the instruction and the case text, and asks for the answer in the fixed shape defined with Pydantic.
5. If the answer has the wrong shape, the gateway tells the model what was wrong and retries a limited number of times. If it still fails, the step is marked failed and the run can resume.
6. Good answers are saved in a local cache, so repeating the same case costs no extra free requests and gives the same result.
7. A short pause between calls keeps the team inside the free rate limit.
8. To switch to Groq, or to try a DeepSeek or GLM model, change the provider and model name in the environment file. No agent changes are needed. Always rerun your test cases after a switch.

5. Which part is AI and which part is code

Your TRD says code should decide wherever code can. The tools are split the same way.

Pipeline step	Tool	Decided by
Read the case into a structure	Gemini plus Pydantic	AI, checked by schema
Compare to the consumer checklist	Gemini	AI against an explicit checklist
Stop early if a critical item is missing	Python orchestrator	Code
Check that a cited section exists	JSON corpus lookup	Code
Check that the section supports the claim	Gemini	AI
Apply the escalation rule	Python	Code
Write the report text	Gemini	AI, from structured results only

Pipeline step	Tool	Decided by
Save every step for tracing	SQLite	Code
Score baseline versus full system	pytest and scoring code	Code

6. Getting started: order of setup

1. Everyone installs Python, Git and OpenCode.
2. One person creates the private GitHub repository and invites the other three.
3. Everyone clones the repository and creates a virtual environment, a private box of packages for this project.
4. Everyone installs the project packages from the requirements file.
5. Each person creates their own Gemini API key. Limits are per project, so do not share one key.
6. Each person makes their own environment file from the example file and checks it is not tracked by Git.
7. Everyone connects OpenCode to a free DeepSeek or GLM model and asks it one simple question.
8. Everyone makes one successful test call to the Gemini model from the project before real work starts.

7. Free resources

Software to download

Need	Where	Notes
Python	python.org	On Windows, tick the option to add Python to PATH
OpenCode	opencode.ai (docs at opencode.ai/docs)	Terminal and desktop app; follow the install page for your system
Node.js (only if installing OpenCode with npm)	nodejs.org	Free; skip if you use another install method
Git	git-scm.com	Also create a free account at github.com
View the SQLite database	sqlitebrowser.org (DB Browser for SQLite)	Lets you open the step log without code
Draw diagrams	mermaid.js.org or draw.io	For the architecture and workflow figures

AI model access

Need	Where	Notes
Gemini API key and your own limits	aistudio.google.com	The rate-limit page shows the real numbers for your project
Gemini documentation	ai.google.dev	Current model names and guides
Backup model provider	console.groq.com	Free tier with its own limits
OpenCode free models (Zen)	opencode.ai/docs/zen	Includes a free DeepSeek V4 Flash and others; the list changes
GLM free models	z.ai and docs.z.ai (pricing page)	GLM-4.7-Flash and GLM-4.5-Flash are free; newer GLM models are paid
DeepSeek (optional, paid by token)	platform.deepseek.com and api-docs.deepseek.com	Very low cost, but not free; check the pricing page
OpenCode Go (optional, paid)	opencode.ai	About 10 dollars a month with DeepSeek and GLM models

Legal source text

Need	Where	Notes
Official Consumer Protection Act, 2019	indiacode.nic.in	Copy the exact text of the chosen sections into the corpus. Do not paraphrase from memory.

Learning

Skill	Where	What to finish
OpenCode	opencode.ai/docs	Getting started, Providers and Models pages
Python basics	cs50.harvard.edu/python or docs.python.org/3/tutorial	Variables, loops, functions, lists, dictionaries, files, imports
Git and GitHub	skills.github.com and git-scm.com/book	Clone, commit, push, branch, pull request
Working with JSON	docs.python.org/3/library/json.html	Load a JSON file and read its contents
Pydantic	docs.pydantic.dev	The Models page
Streamlit	docs.streamlit.io	The Get started tutorial
pytest	docs.pytest.org	Getting started page
SQLite in Python	docs.python.org/3/library/sqlite3.html	Create a table, insert, select
Student extras	education.github.com/pack	GitHub Student Developer Pack, if your college email qualifies. Check its current terms.

8. Free-tier limits and cautions

- **Free model lists change.** The free DeepSeek and GLM options inside OpenCode and on Z.ai may be renamed, limited or removed. Have a second route ready (Section 3).
- **AI-written code needs human checking.** Free coding models make mistakes. Run the tests after every change.
- **Limits change.** Google no longer publishes one fixed table of free limits, and sources disagree. Reports in October 2026 put the Flash models at roughly 10 requests per minute and about 1,500 per day. Check your own numbers in AI Studio before planning.
- **Model names change.** Some older Gemini versions were scheduled to shut down in October 2026. Always copy a current name from AI Studio.
- **Free-tier data may be used by the provider** (Google, and the hosts of DeepSeek and GLM) to improve its products. This is acceptable only because every NyayaSim case is fictional. Never paste a real case.
- **Plan for slow runs.** A full evaluation can take 10 to 15 minutes on the free tier. Test one case at a time while tuning, and rely on the cache.
- **Keep keys private.** If a key reaches GitHub, delete it in AI Studio at once and create a new one.
- **Check terms again** before the final demo, since free tiers and prices change often.

9. What we are not using, and when to add it

Skipped	Why	Add it when
LangGraph, CrewAI, AutoGen	Hide what is happening; the project needs fixed roles and strict rules	Branching becomes hard to manage by hand
PostgreSQL and pgvector	Needs installation; SQLite is enough for one user	Several users or semantic search
Embeddings and vector search	A small corpus needs only exact lookup	Keyword search misses relevant sections
Docker, Redis, task queue	Extra tools with no MVP benefit	Deploying for many users
Langfuse or LangSmith	The SQLite step log already stores every input and output	You want ready-made trace screens
Visual Studio Code or another editor	Replaced by OpenCode as the main tool	A teammate wants a graphical editor to read code. It can be used alongside OpenCode
Local models such as Ollama	You chose a free cloud API	Free limits block you or you need offline use